

11.2 Balanced Search Trees

In the closing of the previous section, we noted that if we could assume a random series of insertions and removals, the standard binary search tree supports $O(\log n)$ expected running times for the basic map operations. However, we may only claim $O(n)$ worst-case time, because some sequences of operations may lead to an unbalanced tree with height proportional to n .

In the remainder of this chapter, we will explore four search-tree algorithms that provide stronger performance guarantees. Three of the four data structures (AVL trees, splay trees, and red-black trees) are based on augmenting a standard binary search tree with occasional operations to reshape the tree and reduce its height.

The primary operation to rebalance a binary search tree is known as a *rotation*. During a rotation, we “rotate” a child to be above its parent, as diagrammed in Figure 11.8.

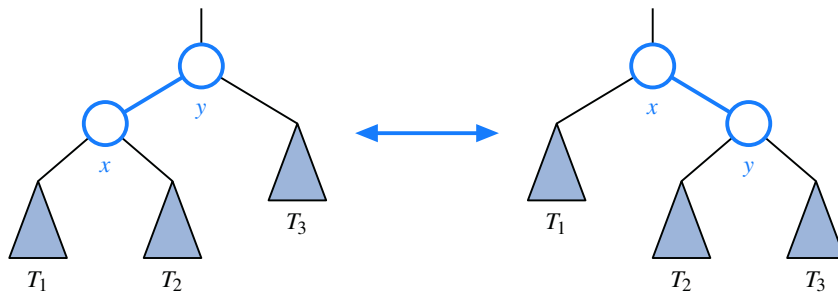


Figure 11.8: A rotation operation in a binary search tree. A rotation can be performed to transform the left formation into the right, or the right formation into the left. Note that all keys in subtree T_1 have keys less than that of position x , all keys in subtree T_2 have keys that are between those of positions x and y , and all keys in subtree T_3 have keys that are greater than that of position y .

To maintain the binary search-tree property through a rotation, we note that if position x was a left child of position y prior to a rotation (and therefore the key of x is less than the key of y), then y becomes the *right* child of x after the rotation, and vice versa. Furthermore, we must relink the subtree of entries with keys that lie between the keys of the two positions that are being rotated. For example, in Figure 11.8 the subtree labeled T_2 represents entries with keys that are known to be greater than that of position x and less than that of position y . In the first configuration of that figure, T_2 is the right subtree of position x ; in the second configuration, it is the left subtree of position y .

Because a single rotation modifies a constant number of parent-child relationships, it can be implemented in $O(1)$ time with a linked binary tree representation.

In the context of a tree-balancing algorithm, a rotation allows the shape of a tree to be modified while maintaining the search-tree property. If used wisely, this operation can be performed to avoid highly unbalanced tree configurations. For example, a rightward rotation from the first formation of Figure 11.8 to the second reduces the depth of each node in subtree T_1 by one, while increasing the depth of each node in subtree T_3 by one. (Note that the depth of nodes in subtree T_2 are unaffected by the rotation.)

One or more rotations can be combined to provide broader rebalancing within a tree. One such compound operation we consider is a **trinode restructuring**. For this manipulation, we consider a position x , its parent y , and its grandparent z . The goal is to restructure the subtree rooted at z in order to reduce the overall path length to x and its subtrees. Pseudocode for a `restructure( $x$ )` method is given in Code Fragment 11.7 and illustrated in Figure 11.9. In describing a trinode restructuring, we temporarily rename the positions x , y , and z as a , b , and c , so that a precedes b and b precedes c in an inorder traversal of T . There are four possible orientations mapping x , y , and z to a , b , and c , as shown in Figure 11.9, which are unified into one case by our relabeling. The trinode restructuring replaces z with the node identified as b , makes the children of this node be a and c , and makes the children of a and c be the four previous children of x , y , and z (other than x and y), while maintaining the inorder relationships of all the nodes in T .

Algorithm `restructure( $x$ )`:

Input: A position x of a binary search tree T that has both a parent y and a grandparent z

Output: Tree T after a trinode restructuring (which corresponds to a single or double rotation) involving positions x , y , and z

- 1: Let (a, b, c) be a left-to-right (inorder) listing of the positions x , y , and z , and let (T_1, T_2, T_3, T_4) be a left-to-right (inorder) listing of the four subtrees of x , y , and z not rooted at x , y , or z .
- 2: Replace the subtree rooted at z with a new subtree rooted at b .
- 3: Let a be the left child of b and let T_1 and T_2 be the left and right subtrees of a , respectively.
- 4: Let c be the right child of b and let T_3 and T_4 be the left and right subtrees of c , respectively.

Code Fragment 11.7: The trinode restructuring operation in a binary search tree.

In practice, the modification of a tree T caused by a trinode restructuring operation can be implemented through case analysis either as a single rotation (as in Figure 11.9a and b) or as a double rotation (as in Figure 11.9c and d). The double rotation arises when position x has the middle of the three relevant keys and is first rotated above its parent, and then above what was originally its grandparent. In any of the cases, the trinode restructuring is completed with $O(1)$ running time.

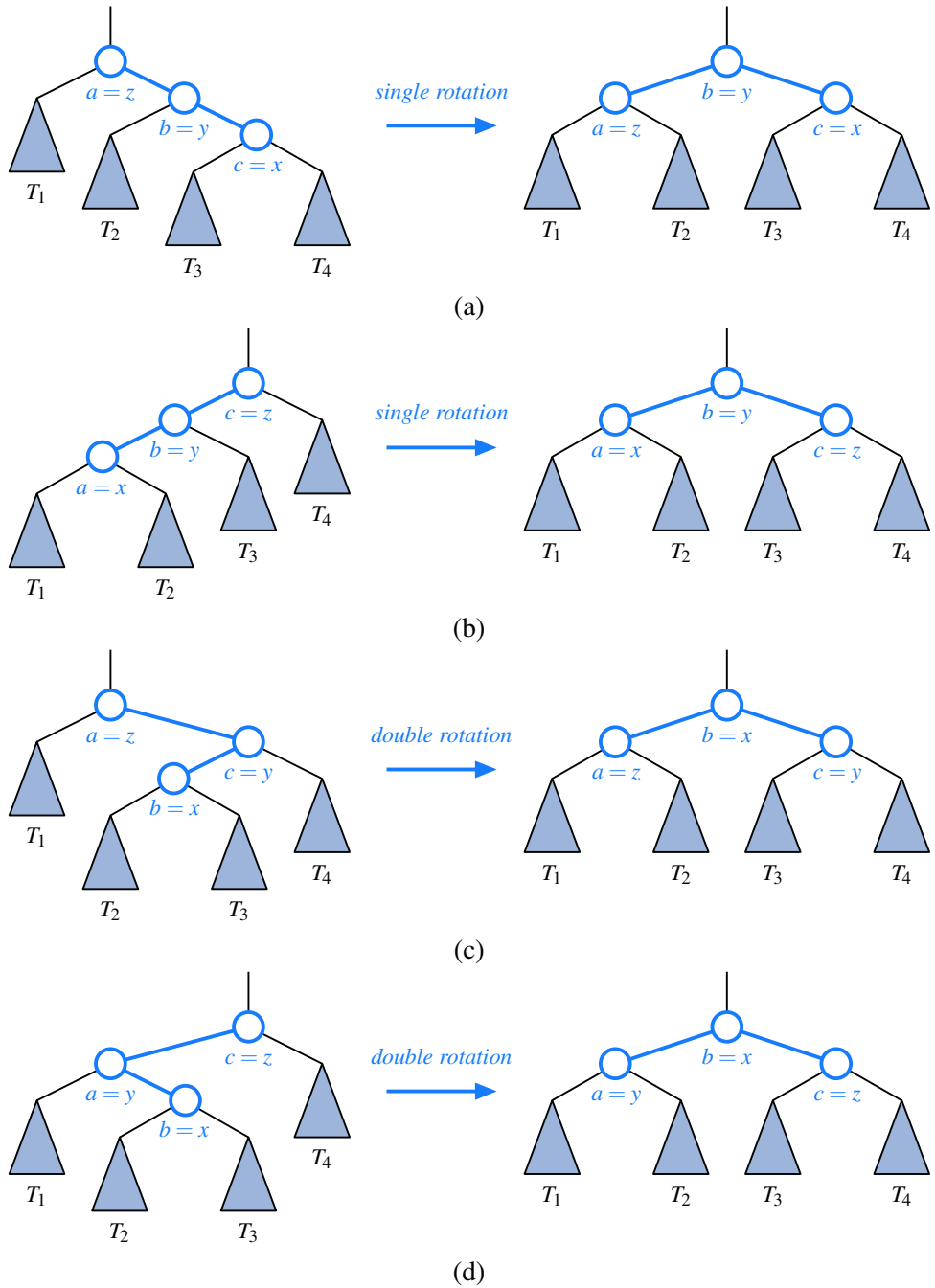


Figure 11.9: Schematic illustration of a trinode restructuring operation: (a and b) require a single rotation; (c and d) require a double rotation.

11.2.1 Java Framework for Balancing Search Trees

Our `TreeMap` class (introduced in Section 11.1.3) is a fully functional map implementation. However, the running time for its operations depend on the height of the tree, and in the worst-case, that height may be $O(n)$ for a map with n entries. Therefore, we have intentionally designed the `TreeMap` class in a way that allows it to be easily extended to provide more advanced tree-balancing strategies. In later sections of this chapter, we will implement subclasses `AVLTreeMap`, `SplayTreeMap`, and `RBTreeMap`. In this section, we describe three important forms of support that the `TreeMap` class offers these subclasses.

Hooks for Rebalancing Operations

Our implementation of the basic map operations in Section 11.1.3 includes strategic calls to three nonpublic methods that serve as *hooks* for rebalancing algorithms:

- A call to `rebalanceInsert( $p$ )` is made from within the `put` method, after a new node is added to the tree at position p (line 61 of Code Fragment 11.4).
- A call to `rebalanceDelete( $p$ )` is made from within the `remove` method, after a node is deleted from the tree (line 88 of Code Fragment 11.4); position p identifies the child of the removed node that was promoted in its place.
- A call to `rebalanceAccess( $p$ )` is made by any call to `get`, `put`, or `remove` that does *not* result in a structural change. Position p , which could be internal or external, represents the deepest node of the tree that was accessed during the operation. This hook is specifically used by the *splay tree* structure (see Section 11.4) to restructure a tree so that more frequently accessed nodes are brought closer to the root.

Within our `TreeMap` class, we provide the trivial declarations of these three methods, having bodies that do nothing, as shown in Code Fragment 11.8. A subclass of `TreeMap` may override any of these methods to implement a nontrivial action to rebalance a tree. This is another example of the *template method design pattern*, as originally discussed in Section 2.3.3.

```
protected void rebalanceInsert(Position<Entry<K,V>> p) { }  
protected void rebalanceDelete(Position<Entry<K,V>> p) { }  
protected void rebalanceAccess(Position<Entry<K,V>> p) { }
```

Code Fragment 11.8: Trivial definitions of `TreeMap` methods that serve as hooks for our rebalancing framework. These methods may be overridden by subclasses in order to perform appropriate rebalancing operations.

Protected Methods for Rotating and Restructuring

To support common restructuring operations, our `TreeMap` class relies on storing the tree as an instance of a new nested class, `BalanceableBinaryTree` (shown in Code Fragments 11.9 and 11.10). That class is a specialization of the original `LinkedBinaryTree` class from Section 8.3.1. This new class provides protected utility methods `rotate` and `restructure` that, respectively, implement a single rotation and a trinode restructuring (described at the beginning of Section 11.2). Although these methods are not invoked by the standard `TreeMap` operations, their inclusion supports greater code reuse, as they are available to all balanced-tree subclasses.

These methods are implemented in Code Fragment 11.10. To simplify the code, we define an additional `relink` utility that properly links parent and child nodes to each other. The focus of the `rotate` method then becomes redefining the relationship between the parent and child, relinking a rotated node directly to its original grandparent, and shifting the “middle” subtree (that labeled as T_2 in Figure 11.8) between the rotated nodes.

For the trinode restructuring, we determine whether to perform a single or double rotation, as originally described in Figure 11.9. The four cases in that figure demonstrate a downward path z to y to x that are respectively right-right, left-left, right-left, and left-right. The first two patterns, with matching orientation, warrant a single rotation moving y upward, while the last two patterns, with opposite orientations, warrant a double rotation moving x upward.

Specialized Nodes with an Auxiliary Data Member

Many tree-balancing strategies require that some form of auxiliary “balancing” information be stored at nodes of a tree. To ease the burden on the balanced-tree subclasses, we choose to add an auxiliary integer value to every node within the `BalanceableSearchTree` class. This is accomplished by defining a new `BSTNode` class, which itself inherits from the nested `LinkedBinaryTree.Node` class. The new class declares the auxiliary variable, and provides methods for getting and setting its value.

We draw attention to an important subtlety in our design, including that of the original `LinkedBinaryTree` subclass. Whenever a low-level operation on an underlying linked tree requires a new node, we must ensure that the correct type of node is created. That is, for our balanceable tree, we need each node to be a `BTNode`, which includes the auxiliary field. However, the creation of nodes occurs within low-level operations, such as `addLeft` and `addRight`, that reside in the original `LinkedBinaryTree` class.

We rely on a technique known as the *factory method design pattern*. The `LinkBinaryTree` class includes a protected method, `createNode` (originally given at lines 30–33 of Code Fragment 8.8), that is responsible for instantiating a new node of the appropriate type. The rest of the code in that class makes sure to always use the `createNode` method when a new node is needed.

In the `LinkBinaryTree` class, the `createNode` method returns a simple `Node` instance. In our new `BalanceableBinaryTree` class, we override the `createNode` method (see lines 22–27 in Code Fragment 11.9), so that a new instance of the `BSTNode` class is returned. In this way, we effectively change the behavior of the low-level operations in the `LinkBinaryTree` class so that it uses instances of our specialized node class, and therefore, that every node in our balanced trees includes support for the new auxiliary field.

```

1  /** A specialized version of LinkBinaryTree with support for balancing. */
2  protected static class BalanceableBinaryTree<K,V>
3      extends LinkBinaryTree<Entry<K,V>> {
4      //----- nested BSTNode class -----
5      // this extends the inherited LinkBinaryTree.Node class
6      protected static class BSTNode<E> extends Node<E> {
7          int aux=0;
8          BSTNode(E e, Node<E> parent, Node<E> leftChild, Node<E> rightChild) {
9              super(e, parent, leftChild, rightChild);
10         }
11         public int getAux() { return aux; }
12         public void setAux(int value) { aux = value; }
13     } //----- end of nested BSTNode class -----
14
15     // positional-based methods related to aux field
16     public int getAux(Position<Entry<K,V>> p) {
17         return ((BSTNode<Entry<K,V>>) p).getAux();
18     }
19     public void setAux(Position<Entry<K,V>> p, int value) {
20         ((BSTNode<Entry<K,V>>) p).setAux(value);
21     }
22     // Override node factory function to produce a BSTNode (rather than a Node)
23     protected
24     Node<Entry<K,V>> createNode(Entry<K,V> e, Node<Entry<K,V>> parent,
25                               Node<Entry<K,V>> left, Node<Entry<K,V>> right) {
26         return new BSTNode<>(e, parent, left, right);
27     }

```

Code Fragment 11.9: The `BalanceableBinaryTree` class, which is nested within the `TreeMap` class definition. (Continues in Code Fragment 11.10.)

```

28  /** Relinks a parent node with its oriented child node. */
29  private void relink(Node<Entry<K,V>> parent, Node<Entry<K,V>> child,
30      boolean makeLeftChild) {
31      child.setParent(parent);
32      if (makeLeftChild)
33          parent.setLeft(child);
34      else
35          parent.setRight(child);
36  }
37  /** Rotates Position p above its parent. */
38  public void rotate(Position<Entry<K,V>> p) {
39      Node<Entry<K,V>> x = validate(p);
40      Node<Entry<K,V>> y = x.getParent();           // we assume this exists
41      Node<Entry<K,V>> z = y.getParent();           // grandparent (possibly null)
42      if (z == null) {
43          root = x;                               // x becomes root of the tree
44          x.setParent(null);
45      } else
46          relink(z, x, y == z.getLeft());           // x becomes direct child of z
47      // now rotate x and y, including transfer of middle subtree
48      if (x == y.getLeft()) {
49          relink(y, x.getRight(), true);           // x's right child becomes y's left
50          relink(x, y, false);                     // y becomes x's right child
51      } else {
52          relink(y, x.getLeft(), false);           // x's left child becomes y's right
53          relink(x, y, true);                       // y becomes left child of x
54      }
55  }
56  /** Performs a trinode restructuring of Position x with its parent/grandparent. */
57  public Position<Entry<K,V>> restructure(Position<Entry<K,V>> x) {
58      Position<Entry<K,V>> y = parent(x);
59      Position<Entry<K,V>> z = parent(y);
60      if ((x == right(y)) == (y == right(z))) {     // matching alignments
61          rotate(y);                               // single rotation (of y)
62          return y;                                 // y is new subtree root
63      } else {                                       // opposite alignments
64          rotate(x);                               // double rotation (of x)
65          rotate(x);
66          return x;                                 // x is new subtree root
67      }
68  }
69  }

```

Code Fragment 11.10: The `BalanceableBinaryTree` class, which is nested within the `TreeMap` class definition (continued from Code Fragment 11.9).